

CAD DATA LABELLING PROJECT PROPOSAL

A Comprehensive Technical Framework for 3D CAD Model Annotation,
Machine Learning Data Preparation, and Feature Recognition Training

Prepared for:

Isidor AI

Frontier Enterprise AI | San Francisco & London

Date: January 23, 2026

Version: 2.0 (Expanded Technical Edition)

Classification: Confidential

1. EXECUTIVE SUMMARY

This proposal presents a comprehensive technical framework for developing a CAD (Computer-Aided Design) data labelling initiative tailored to Isidor's mission of transforming raw, unstructured proprietary enterprise data into reinforcement-learning-ready curriculum for AI model training. The project addresses the critical industry need for high-quality, annotated 3D CAD datasets that can power next-generation machine learning models in manufacturing automation, design synthesis, and engineering applications.

CAD data represents one of the most valuable yet underutilised sources of enterprise data. The global data annotation and labelling market is projected to grow from USD 0.8 billion in 2022 to USD 3.6 billion by 2027 (CAGR 33.2%), with CAD-specific labelling emerging as a high-value niche. Unlike image or text annotation, CAD labelling requires specialised domain expertise in engineering, manufacturing processes, and geometric reasoning—creating significant barriers to entry and competitive differentiation.

1.1 Key Technical Challenges Addressed

- **Boundary Representation (B-Rep) Complexity:** CAD models store geometry as faces, edges, and vertices with complex topological relationships that must be preserved during labelling to enable downstream graph neural network (GNN) training.
- **Feature Intersection Handling:** Unlike simple object detection, machining features (holes, pockets, slots) often intersect, creating ambiguous boundaries that require expert judgment and consistent labelling protocols.
- **Multi-Resolution Requirements:** Different ML architectures require different input formats—voxels for 3D CNNs, point clouds for PointNet++, face adjacency graphs for GNNs—necessitating flexible preprocessing pipelines.
- **Scarcity of Annotated Data:** While millions of CAD models exist in enterprise PLM systems, professionally labelled datasets remain scarce. The MFCAD++ dataset (59,665 models) represents the current state-of-the-art, highlighting the opportunity for Isidor.
- **Domain Expertise Requirements:** Labellers must understand manufacturing processes, GD&T; (Geometric Dimensioning and Tolerancing), and engineering conventions to produce accurate annotations.

2. ABOUT ISIDOR AND STRATEGIC ALIGNMENT

Isidor is a frontier enterprise AI company headquartered in San Francisco and London, developing secure, vertically-integrated pipelines for enterprise data. The company specialises in transforming raw, unstructured proprietary data into reinforcement-learning-ready curriculum for model training. Backed by leading investors including Future Present, Gradient, and Seedcamp, with advisors from JPMorgan Chase, Google DeepMind, Amazon, Perplexity, Microsoft, KKR, Apple, AMD, and other industry leaders, Isidor is positioned at the cutting edge of enterprise AI infrastructure.

2.1 Isidor's Core Capabilities and CAD Project Alignment

Isidor's five-pillar capability framework maps directly to the requirements of a CAD data labelling project:

- **Evaluation (SOTA Benchmarks):** CAD labelling will generate industry-specific benchmarks for manufacturing AI. Current benchmarks like MFCAD++ contain 59,665 models with 25 machining feature classes. Isidor can create proprietary benchmarks spanning additional industries (aerospace, medical devices, automotive) with expanded feature taxonomies (50+ classes). These benchmarks enable rigorous evaluation of client AI models against domain-specific standards.
- **Verifiers (Post-Processing for Gen AI):** Labelled CAD data enables verification of generative design outputs. When AI systems generate new part designs, verifiers can validate that generated geometry contains valid machining features, correct tolerances, and manufacturable specifications. The labelled dataset provides ground truth for automated quality checks.
- **Reasoning Trace Data:** CAD feature recognition captures the implicit reasoning of design engineers. By labelling not just features but the relationships between features (e.g., 'this hole is a mounting point for bolt X'), Isidor captures the semantic reasoning that engineers apply. This reasoning trace data is invaluable for training AI systems that must understand design intent.
- **Foundation Model Post-Training:** Labelled CAD datasets enable fine-tuning of foundation models for manufacturing domains. A 3D foundation model pre-trained on general shapes can be fine-tuned on Isidor's labelled machining feature data to achieve state-of-the-art performance on manufacturing-specific tasks.
- **Security and Privacy:** CAD data contains proprietary design IP requiring enterprise-grade protection. Isidor's existing security infrastructure (end-to-end encryption, access controls, audit logging) ensures client CAD data remains confidential throughout the labelling pipeline.

3. UNDERSTANDING CAD FILE FORMATS AND DATA STRUCTURES

A deep understanding of CAD file formats is essential for designing an effective labelling pipeline. CAD files are fundamentally different from images or text—they contain rich geometric, topological, and semantic information that must be preserved and leveraged during annotation.

3.1 Boundary Representation (B-Rep) Fundamentals

B-Rep is the de facto standard for representing solid models in professional CAD systems (CATIA, SolidWorks, NX, Creo). Understanding B-Rep is critical because it defines what information is available for labelling and how features can be identified:

- **Vertices (0D entities):** Points in 3D space where edges meet. Each vertex has (x, y, z) coordinates. In a simple cube, there are 8 vertices. Vertices define the corners of features like rectangular pockets or the centres of holes.
- **Edges (1D entities):** Curves connecting vertices. Edges can be lines (straight), arcs (circular segments), or NURBS curves (free-form). Each edge has a start vertex, end vertex, and curve geometry. Edges define the boundaries between faces and are critical for identifying feature outlines.
- **Faces (2D entities):** Bounded surfaces forming the outer shell of the solid. Faces can be planar, cylindrical, conical, spherical, toroidal, or B-spline. Each face has: (1) a surface geometry defining its shape, (2) one or more loops of edges defining its boundary, and (3) a normal vector pointing outward. Faces are the primary unit of labelling—each face is assigned a feature class.
- **Coedges (Oriented Edges):** Each edge appears twice in B-Rep—once for each adjacent face, with opposite orientations. Coedges maintain the topological consistency of the model and enable traversal of adjacent faces. The coedge structure is essential for graph-based ML representations.
- **Shells and Solids:** A shell is a closed set of faces forming a watertight boundary. A solid is one or more shells defining a physical volume. Most machined parts are single-shell solids, but assemblies contain multiple solids.

3.2 Native vs Neutral File Formats

CAD files fall into two categories, each with implications for the labelling pipeline:

Native Formats (Proprietary):

- **CATIA (.CATPart, .CATProduct):** Dassault Systèmes' format used extensively in aerospace and automotive. Contains full parametric history, constraints, and feature tree. Requires CATIA license to read.
- **SolidWorks (.SLDPRT, .SLDASM):** Popular in mid-market manufacturing. Contains feature tree, configurations, and design intent. Requires SolidWorks API for extraction.
- **Siemens NX (.prt):** Used in automotive and industrial machinery. Contains PMI, CAM data, and simulation results. Requires NX Open API.
- **Creo/Pro-E (.prt):** PTC's format common in consumer products. Contains mechanism data, family tables, and relations.
- **Inventor (.ipt, .iam):** Autodesk's mechanical CAD format. Contains iFeatures, iParts, and frame generator data.

Challenge: Native formats preserve design intent (parametric features, constraints, feature order) but require expensive software licenses and format-specific parsing code. For a labelling operation serving multiple industries, supporting all native formats is impractical.

Neutral Formats (Interoperable):

- **STEP AP242 (.stp, .step):** ISO 10303 standard. The recommended format for CAD labelling. Preserves complete B-Rep geometry, PMI (Product Manufacturing Information), GD&T, assembly structure, and material assignments. Supported by all major CAD systems. AP242 (2014) supersedes AP203 and AP214, adding model-based definition (MBD) support.
- **STEP AP203 (.stp):** Earlier STEP version. Preserves geometry, topology, and configuration management but lacks colour, layers, and GD&T.; Still widely used for legacy compatibility.
- **IGES (.igs, .iges):** Initial Graphics Exchange Specification (1980). The oldest neutral format, still supported but deprecated. Preserves surface geometry but often introduces gaps, missing faces, and surface orientation errors during translation. Not recommended for precision labelling.
- **Parasolid (.x_t, .x_b):** Siemens' geometric kernel format. Excellent geometric fidelity for solid models. Text (.x_t) and binary (.x_b) variants. Used internally by SolidWorks, NX, and Solid Edge.
- **JT (.jt):** Siemens' lightweight format for visualisation and PLM. Contains tessellated geometry, PMI, and metadata. Good for assembly-level labelling but loses precise B-Rep data.
- **STL (.stl):** Stereolithography format. Represents surfaces as triangular meshes only—no topology, no features, no colours. Suitable for 3D printing and visualisation but loses critical information for feature-level labelling. Use only when converting from higher-fidelity formats with controlled tessellation parameters.

3.3 File Format Comparison for Data Labelling

Format	B-Rep	PMI/GD&T	Assembly	Colour	Labelling Use
STEP AP242	✓ Full	✓ Full	✓ Full	✓	Excellent - Primary
STEP AP203	✓ Full	✗	✓ Basic	✗	Good - Legacy
Parasolid	✓ Full	✗	✓ Full	✓	Excellent
IGES	~ Partial	✗	✗	✗	Poor - Avoid
JT	~ Tessellated	✓ Full	✓ Full	✓	Moderate - Assembly
STL	✗ Mesh only	✗	✗	✗	Limited - Voxel input
OBJ	✗ Mesh only	✗	✗	✓	Limited - Rendering

Table 1: File format capabilities for CAD data labelling

Recommendation: Standardise on STEP AP242 as the primary input format. For clients unable to export AP242, accept Parasolid (.x_t) as secondary. Convert legacy IGES files to STEP using commercial translators (e.g., CADfix, TransMagic) with geometry healing before labelling.

4. FILES AND DATA REQUIRED FOR LABELLING

A comprehensive CAD data labelling project requires multiple file types and metadata sources to create complete, high-quality training datasets. This section details every input required.

4.1 Primary 3D Model Files

Each CAD model to be labelled must include the following primary data:

- **STEP/Parasolid File (Required):** The authoritative geometric representation. Must be a valid, watertight solid (no gaps, no self-intersections). File should be exported with maximum precision settings (typically 0.001mm chord tolerance). For assemblies, include both the assembly file and individual part files.
- **Tessellated Mesh (Generated):** An STL or OBJ mesh derived from the B-Rep with controlled resolution. Mesh quality directly impacts voxelisation and point cloud sampling. Recommended settings: maximum edge length 1mm, angular deviation 15°, chord height 0.1mm. This is generated during preprocessing, not provided by the client.
- **Multi-View Rendered Images (Generated):** 2D renders of the 3D model from standardised viewpoints (front, back, left, right, top, bottom, isometric). Resolution: minimum 512×512 pixels. Format: PNG with transparent background. Used for labeller reference and for training vision-language models (VLMs) that combine image and CAD understanding.
- **Native CAD File (Optional):** If available, the original native format preserves feature history and design intent. Useful for extracting feature trees programmatically and for validating labeller interpretations against original designer intent.

4.2 Metadata Requirements

Comprehensive metadata enables consistent labelling, quality control, and downstream ML applications:

- **Unique Identifier:** A globally unique ID for each model (e.g., UUID v4). Format: 'isidor_cad_xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx'. This ID links all associated files, labels, and audit trails.
- **Part Classification:** Industry sector (aerospace, automotive, medical, consumer), part family (bracket, housing, shaft, gear), functional role (structural, cosmetic, wear component). Enables stratified sampling and domain-specific model training.
- **Geometric Properties (Computed):** Bounding box dimensions (length, width, height in mm), total volume (mm³), total surface area (mm²), centre of mass coordinates, moments of inertia. These normalised metrics enable size-independent feature learning.
- **Topological Metrics (Computed):** Number of faces, number of edges, number of vertices, Euler characteristic, number of shells, number of loops per face. These metrics indicate model complexity and guide labelling time estimates.
- **Manufacturing Context:** Intended manufacturing process (CNC milling, turning, casting, additive), material class (aluminium, steel, titanium, polymer), tolerance grade (ISO IT5-IT14). Enables manufacturing-aware labelling decisions.
- **Feature Inventory (Pre-computed):** Counts of detected feature types from rule-based pre-processing. Example: '4 through-holes, 2 pockets, 8 fillets'. Serves as a checklist for labellers and enables stratified sampling.
- **Provenance Data:** Source CAD software and version, export date, original filename, data provider, license/usage rights. Essential for compliance, debugging, and reproducing results.
- **Quality Flags:** Automated checks for: watertight solid, consistent face normals, no self-intersections, no degenerate faces (area < threshold), no short edges (length < threshold). Models failing checks require manual review before labelling.

4.3 Supplementary Reference Materials

- **2D Engineering Drawings (If Available):** PDF or DXF drawings with dimensions, tolerances, and GD&T; callouts. Critical for resolving ambiguous features (e.g., distinguishing a press-fit hole from a clearance hole based on tolerance). Labellers should cross-reference drawings when available.
- **Bill of Materials (BOM):** For assemblies, a structured BOM listing all components, quantities, and relationships. Enables assembly-level labelling (mate relationships, kinematic joints).
- **Design Specification Documents:** Functional requirements, material specifications, surface finish requirements. Provides context for labelling design intent.
- **Reference Images:** Photos of manufactured parts, machining setups, or similar designs. Helps labellers understand real-world manufacturing context.

5. PROJECT STRUCTURE AND DATA PIPELINE

5.1 End-to-End Data Pipeline Architecture

The CAD labelling pipeline transforms raw client CAD files into ML-ready annotated datasets through six stages. Each stage includes detailed processes, quality gates, and output specifications:

Stage 1: Data Ingestion and Validation (Days 1-3 per batch)

- **File Collection:** Receive CAD files via secure upload (SFTP, S3 with client-side encryption, or API). Support batch uploads of 100-10,000 files. Validate file integrity (checksum verification, virus scanning).
- **Format Detection:** Automatically identify file format from extension and magic bytes. Log format distribution for pipeline optimisation.
- **Format Conversion:** Convert non-STEP formats to STEP AP242 using Open Cascade Technology (OCCT) or commercial translators. Log conversion warnings (e.g., 'IGES surface healed', 'Tolerance relaxed').
- **Geometric Validation:** Run automated checks: (1) solid is watertight (Euler characteristic = 2 for simple solids), (2) no self-intersections, (3) consistent face orientations (normals pointing outward), (4) no degenerate geometry (zero-area faces, zero-length edges). Reject or flag models failing critical checks.
- **Metadata Extraction:** Compute geometric properties (bounding box, volume, surface area), topological metrics (face/edge/vertex counts), and generate unique IDs. Store in structured database (PostgreSQL with PostGIS for spatial queries).
- **Output:** Validated STEP files in standardised directory structure, metadata records in database, quality report summarising batch statistics.

Stage 2: Preprocessing for ML Representations (Days 2-5 per batch)

- **Tessellation:** Convert B-Rep surfaces to triangular meshes using OCCT BRepMesh. Parameters: linear deflection 0.1mm, angular deflection 0.5 radians. Output: indexed mesh with vertex positions, face indices, and vertex normals. Store as OBJ or binary PLY for efficient loading.
- **Voxelisation:** Convert solid to 3D occupancy grid using binvox or custom ray-casting. Standard resolutions: 32^3 (fast preview), 64^3 (training default), 128^3 (high-fidelity). Normalise model to unit cube before voxelising. Output: HDF5 array with shape (resolution, resolution, resolution) and dtype uint8 (0=empty, 1=occupied).
- **Point Cloud Sampling:** Sample N points uniformly from mesh surface using Poisson disk sampling or farthest point sampling (FPS). Standard sizes: 1024 (lightweight), 2048 (default), 4096 (high-density). For each point, store: (x, y, z) position, (nx, ny, nz) surface normal. Output: NumPy array shape (N, 6).
- **Face Adjacency Graph (AAG) Construction:** Build graph where nodes=faces, edges=shared boundaries. For each node (face), store: face_id, surface_type (plane/cylinder/cone/sphere/torus/bspline), area, centroid, normal vector, principal curvatures. For each edge (adjacency), store: edge_id, convexity (convex/concave/smooth), dihedral angle, edge length. Output: DGL or PyTorch Geometric graph object.
- **UV-Grid Extraction:** For each face, sample a regular grid in the parametric (u,v) domain. Grid size: typically 10×10 or 20×20 points. For each grid point, store: (x,y,z) global coordinates, (nx,ny,nz) normal, visibility mask (1=on face, 0=trimmed). This representation (from UV-Net) enables CNN-based learning on faces. Output: NumPy array shape (num_faces, grid_u, grid_v, 7).
- **Scale Normalisation:** Transform all representations to a canonical scale. Common approaches: (1) fit to unit bounding box, (2) normalise by diagonal length, (3) normalise by $\text{volume}^{1/3}$. Record original scale factor for inverse transformation. Consistent scaling is critical—models must be comparable across the dataset.
- **Data Augmentation (Training Only):** Generate augmented versions: random rotations (SO(3) sampling), random scaling (0.8-1.2 $\times$), random jittering (Gaussian noise $\sigma=0.01$). Augmentation increases effective dataset size and improves model generalisation. Do not augment validation/test sets.

Stage 3: Annotation Platform Setup (Days 3-7 initial setup)

- **3D Viewer Integration:** Deploy web-based 3D viewer capable of loading STEP files directly (using OCCT WebAssembly or Three.js with STEP loader). Viewer must support: orbit/pan/zoom navigation, face selection (click to select face), multi-face selection (shift-click or lasso), section views (clipping planes), measurement tools (distance, angle), screenshot capture.
- **Labelling Interface:** Alongside the 3D viewer, display: (1) taxonomy tree for feature selection, (2) per-face label assignment panel, (3) metadata display panel showing computed properties, (4) notes field for labeller comments, (5) confidence slider (1-5 scale for uncertain cases). Interface must support keyboard shortcuts for common labels (e.g., 'h' for hole, 'p' for pocket).
- **Task Management:** Implement task queue with: prioritisation (urgent, normal, backlog), assignment rules (round-robin, skill-based), progress tracking (tasks completed, in-progress, pending), deadline management. Each task = one CAD model + all associated files.
- **Collaboration Features:** Enable escalation workflow (labeller flags uncertain model → senior reviewer resolves), comment threads on specific faces, version history of label changes, inter-annotator agreement tracking (assign same model to multiple labellers for QA samples).

Stage 4: Labelling Execution (Ongoing)

- **Face-Level Labelling:** The primary task: assign one feature class to each face in the B-Rep model. Example: Face #23 → 'through_hole_cylindrical'. Labeller clicks on face in 3D viewer, selects label from taxonomy, confirms. Repeat for all faces. Unlabelled faces default to 'stock' (the base material).
- **Feature Instance Grouping:** After labelling individual faces, group faces into feature instances. Example: Faces #23, #24, #25 together form 'Hole_Instance_1'. This enables instance segmentation training (distinguishing individual holes vs. just 'there are holes somewhere').
- **Relationship Annotation (Advanced):** For high-value datasets, annotate feature relationships: 'Hole_1 is coaxial with Hole_2', 'Pocket_1 is symmetric to Pocket_2', 'Fillet_3 blends Edge_7 and Edge_8'. Relationship labels enable reasoning-aware AI models.
- **Confidence Scoring:** For each label, record labeller confidence (1=uncertain, 5=certain). Low-confidence labels trigger review. Confidence metadata enables training with label uncertainty.
- **Time Tracking:** Log time per model (automatic via platform). Typical rates: simple parts (10-20 faces) = 2-5 minutes, medium parts (50-100 faces) = 10-20 minutes, complex parts (200+ faces) = 30-60 minutes. Time data informs pricing and capacity planning.

Stage 5: Quality Assurance (Parallel to Stage 4)

- **Automated Checks:** Run programmatic validation: (1) all faces labelled (no orphans), (2) labels are valid taxonomy entries, (3) feature instances are connected (faces in same instance share edges), (4) geometric consistency (cylindrical faces labelled as holes have circular cross-sections). Flag violations for review.
- **Sampling-Based Review:** Senior reviewer examines random sample (10-20%) of completed models. Assess: label accuracy, instance grouping correctness, consistency with guidelines. Record errors by type (mislabel, missed feature, wrong instance boundary).
- **Inter-Annotator Agreement (IAA):** Assign 5-10% of models to multiple independent labellers. Compute agreement metrics: Cohen's Kappa (κ) for face-level labels, Intersection-over-Union (IoU) for instance boundaries. Target: $\kappa > 0.85$ for mature taxonomy. Low IAA indicates ambiguous guidelines—revise and retrain.
- **Feedback Loop:** Route QA findings to labellers as feedback. Track individual labeller accuracy over time. Provide additional training for labellers with high error rates. Celebrate and reward high-performing labellers.
- **Gold Standard Maintenance:** Maintain a set of 50-100 'gold' models with expert-verified labels. Periodically test labellers against gold set to detect drift. Update gold set as taxonomy evolves.

Stage 6: Export and Delivery (Per release)

- **Dataset Packaging:** Assemble final dataset with directory structure: /step/ (original STEP files), /mesh/ (tessellated OBJ files), /voxel/ (HDF5 voxel grids), /pointcloud/ (NumPy point clouds), /graphs/ (DGL/PyG graph files), /labels/ (JSON annotation files), /splits/ (train.txt, val.txt, test.txt). Include README with format specifications.
- **Label Format:** Export labels in standardised JSON schema (see Section 6 for detailed example). Include both face-level labels and instance-level groupings. Provide conversion scripts for common ML frameworks (PyTorch, TensorFlow).
- **Split Generation:** Create train/val/test splits (typically 70/15/15 or 80/10/10). Ensure stratified sampling: each split has similar distribution of feature types, complexity levels, and industry sectors. Publish split files listing model IDs in each set.
- **Documentation:** Include: taxonomy definition document (all feature classes with descriptions and examples), labelling guidelines document (rules and edge cases), data card (dataset statistics, intended use, limitations), model card template (for downstream users to document their models).
- **Delivery:** Transfer via secure channel (encrypted S3, SFTP). Provide checksums for integrity verification. Offer optional cloud-hosted access (private S3 bucket, BigQuery table) for large datasets.

5.2 Feature Labelling Taxonomy

The labelling taxonomy defines the classification schema for CAD features. Based on industry standards (ISO 10303, STEP-NC) and existing research datasets (MFCAD++, FeatureNet), we propose a hierarchical 25-class taxonomy for machining features, extensible to additional manufacturing domains:

Class ID	Feature Name	Description	Typical Faces
0	stock	Base material not part of any feature	1-6
1	through_hole	Cylindrical hole passing completely through	1-2
2	blind_hole	Cylindrical hole with flat or conical bottom	2-3
3	countersink	Conical recess at hole entry	1
4	counterbore	Cylindrical recess at hole entry	2-3
5	rectangular_pocket	Rectangular depression with flat bottom	5-9
6	rectangular_through_slot	Rectangular channel passing through	3-5
7	rectangular_blind_slot	Rectangular channel with closed end	4-6
8	rectangular_passage	Rectangular opening through material	4
9	rectangular_step	L-shaped removal at edge	2-3
10	triangular_pocket	Triangular depression	4-6
11	triangular_passage	Triangular opening through	3
12	circular_pocket	Circular depression	2-3
13	circular_through_slot	Circular arc channel through	2-4
14	six_sided_pocket	Hexagonal depression	7-9
15	six_sided_passage	Hexagonal opening through	6
16	o_ring_groove	Circular groove for O-ring seal	3
17	chamfer	Angled edge transition	1
18	fillet	Rounded edge transition (concave)	1
19	round	Rounded edge transition (convex)	1
20	boss	Cylindrical protrusion	1-2
21	rib	Thin wall protrusion for reinforcement	2-5
22	taper	Conical surface transition	1
23	thread	Helical thread (internal or external)	1
24	keyway	Slot for key in shaft	3-5

Table 2: 25-class machining feature taxonomy based on MFCAD++ standard

6. LABELLED DATA FORMAT SPECIFICATION

This section presents the complete JSON schema for labelled CAD data, followed by a concrete example based on a simple bracket part with holes and pockets. This format is compatible with state-of-the-art CAD feature recognition frameworks (Hierarchical CADNet, BRepNet, AAGNet).

6.1 JSON Schema Overview

Each labelled CAD model produces a JSON file with the following top-level structure:

```
{
  "metadata": { ... },          // Model identification and properties
  "geometry_summary": { ... },  // Computed geometric metrics
  "faces": [ ... ],            // Per-face labels and attributes
  "feature_instances": [ ... ], // Grouped features
  "relationships": [ ... ],    // Inter-feature relationships (optional)
  "labelling_info": { ... }    // Provenance and quality metadata
}
```

6.2 Complete Example: Mounting Bracket with Holes

The following example shows a fully labelled mounting bracket part. This bracket has a rectangular base with 4 mounting holes, 1 central pocket, and 8 edge fillets—a typical industrial part. The STEP file would be named 'bracket_001.step' and the label file 'bracket_001.json':

```
{
  "metadata": {
    "model_id": "isidor_cad_a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "filename": "bracket_001.step",
    "part_name": "Mounting Bracket Assembly",
    "part_family": "bracket",
    "industry_sector": "industrial_machinery",
    "material_class": "aluminium_6061",
    "manufacturing_process": "cnc_milling",
    "source_cad_software": "SolidWorks 2023",
    "step_schema": "AP242",
    "units": "millimeters"
  },

  "geometry_summary": {
    "bounding_box": {
      "min": [0.0, 0.0, 0.0],
      "max": [120.0, 80.0, 15.0],
      "dimensions": [120.0, 80.0, 15.0]
    },
    "volume_mm3": 112500.0,
    "surface_area_mm2": 24800.0,
    "center_of_mass": [60.0, 40.0, 7.5],
    "num_faces": 42,
    "num_edges": 98,
    "num_vertices": 58,
    "euler_characteristic": 2,
    "is_watertight": true,
  }
}
```

```

    "has_consistent_normals": true
  },

  "faces": [
    {
      "face_id": 0,
      "surface_type": "plane",
      "label": "stock",
      "label_id": 0,
      "instance_id": null,
      "area_mm2": 9600.0,
      "centroid": [60.0, 40.0, 15.0],
      "normal": [0.0, 0.0, 1.0],
      "is_top_face": true
    },
    {
      "face_id": 1,
      "surface_type": "plane",
      "label": "stock",
      "label_id": 0,
      "instance_id": null,
      "area_mm2": 9600.0,
      "centroid": [60.0, 40.0, 0.0],
      "normal": [0.0, 0.0, -1.0],
      "is_bottom_face": true
    },
    {
      "face_id": 2,
      "surface_type": "cylinder",
      "label": "through_hole",
      "label_id": 1,
      "instance_id": "hole_1",
      "area_mm2": 188.5,
      "centroid": [15.0, 15.0, 7.5],
      "normal": null,
      "axis": [0.0, 0.0, 1.0],
      "radius": 4.0,
      "hole_diameter": 8.0
    },
    {
      "face_id": 3,
      "surface_type": "cylinder",
      "label": "through_hole",
      "label_id": 1,
      "instance_id": "hole_2",
      "area_mm2": 188.5,
      "centroid": [105.0, 15.0, 7.5],
      "normal": null,
      "axis": [0.0, 0.0, 1.0],
      "radius": 4.0,
      "hol

```

(JSON example continues - see full specification in delivered dataset)

7. DATA PREPROCESSING AND SIMPLIFICATION

Raw CAD data requires extensive preprocessing before ML model training. This section details each simplification step with technical specifications and rationale, drawing on established practices from ML preprocessing pipelines.

7.1 Geometric Simplifications

- **Defeaturing (Feature Suppression):** Small features that don't contribute to the learning objective should be removed. Implementation: Identify faces with area $<$ threshold (typically 1mm^2 or 0.1% of total surface area). Identify short edges (length $<$ 0.5mm). Use CAD kernel operations (OCCT ShapeUpgrade) to remove small fillets, chamfers, logos, and text. *Rationale:* Small features create noise in voxel/point cloud representations and distract from primary machining features. They also create very small mesh triangles that degrade numerical stability.
- **Tessellation Quality Control:** B-Rep surfaces must be converted to triangular meshes for most ML pipelines. Implementation: Set chord deviation (maximum distance from true surface) to 0.01-0.1mm depending on model size. Set angular deviation to 0.5 radians ($\approx 30^\circ$) for curved surfaces. Ensure minimum edge length $>$ 0.1mm to avoid degenerate triangles. Target mesh quality: aspect ratio $<$ 10, minimum angle $>$ 15° . *Rationale:* Poor tessellation creates artifacts in voxelisation and point cloud sampling. Consistent tessellation parameters ensure dataset homogeneity.
- **Topology Repair:** Real-world CAD files often contain topological errors from format conversion or legacy designs. Implementation: Detect and heal gaps between faces (tolerance 0.001mm). Fix inconsistent face orientations (normals pointing inward instead of outward). Remove duplicate faces and edges. Stitch open shells into closed solids where possible. Use CAD healing tools (CADfix, OCCT ShapeFix). *Rationale:* Topological errors cause voxelisation failures, invalid face adjacency graphs, and undefined normals.
- **Scale Normalisation:** Models span orders of magnitude in size (watchparts: 1mm, aircraft: 10m). ML models expect consistent input scales. Implementation: Compute bounding box diagonal $d = \sqrt{L^2 + W^2 + H^2}$. Scale model by $1/d$ to fit in unit sphere. Alternatively, scale to unit bounding box (divide by max dimension). Record scale factor for inverse transformation. Apply same normalisation to all representations (mesh, voxel, point cloud). *Rationale:* Normalisation enables scale-invariant feature learning and prevents numerical overflow/underflow in neural network computations.
- **Orientation Alignment:** Random model orientations create unnecessary variation. Implementation: Compute principal component analysis (PCA) of vertex positions. Align principal axes with coordinate axes (largest variance along Z, second along Y). Alternatively, use feature-based alignment (e.g., largest planar face aligned with XY plane). Apply rotation to all representations consistently. *Rationale:* Canonical orientation reduces the space of possible transformations the model must learn, improving generalisation with less data.
- **Assembly Decomposition:** Assemblies contain multiple parts that must be processed separately for part-level labelling. Implementation: Parse assembly structure from STEP file. Extract each solid body as separate STEP file. Preserve assembly hierarchy metadata (parent-child relationships, mate constraints). Generate assembly-level labels linking parts to their roles. *Rationale:* Part-level labelling is more tractable than assembly-level. Assembly relationships are captured in metadata for downstream assembly-aware models.

7.2 Feature Extraction for ML Representations

Different ML architectures require different input representations. The preprocessing pipeline generates multiple representations from each normalised model:

- **Voxel Grid Generation:** Convert solid to 3D occupancy grid using ray-casting or surface intersection. Process: (1) Create resolution³ grid of empty voxels. (2) For each voxel center, test if point is inside solid (using ray-casting or winding number). (3) Mark voxel as occupied (1) or empty (0). Store as dense NumPy array or sparse representation. Standard resolutions: 32^3 (262K voxels), 64^3 (2.1M voxels), 128^3 (16.8M)

voxels). *Use case:* 3D CNNs (FeatureNet, VoxNet), occupancy networks.

- **Point Cloud Sampling:** Sample points uniformly from mesh surface. Process: (1) Compute area of each mesh triangle. (2) Sample triangles with probability proportional to area. (3) For each sampled triangle, generate random point using barycentric coordinates. (4) Compute surface normal at sampled point (interpolate from vertex normals). Output: (N, 6) array with (x, y, z, nx, ny, nz). Sampling methods: uniform random, Poisson disk (minimum spacing), farthest point sampling (FPS, maximum coverage). *Use case:* PointNet, PointNet++, DGCNN.

- **Face Adjacency Graph (AAG) Construction:** Build graph capturing B-Rep topology. Process: (1) Create node for each face in B-Rep. (2) Create edge for each pair of faces sharing a boundary edge. (3) For each node, compute features: surface_type (one-hot encoded), area, centroid, normal, principal curvatures (κ_1 , κ_2), bounding box. (4) For each edge, compute features: shared_edge_length, dihedral_angle, convexity (convex/concave/smooth). Store as DGL or PyTorch Geometric Data object. *Use case:* Graph Neural Networks (Hierarchical CADNet, BRepNet, AAGNet).

- **UV-Grid Extraction:** Sample faces in parametric domain for CNN-compatible representation. Process: (1) For each face, define (u, v) parameter bounds. (2) Create regular grid (e.g., 10×10) in parameter space. (3) For each grid point, evaluate: 3D coordinates (x, y, z), surface normal (nx, ny, nz), trimming mask (1 if on face, 0 if in trimmed region). (4) Stack face grids into tensor shape (num_faces, grid_u, grid_v, 7). *Use case:* UV-Net, face-level CNNs.

- **Multi-View Rendering:** Generate 2D images from standardised viewpoints. Process: (1) Set up virtual camera at 6 orthographic positions ($\pm X$, $\pm Y$, $\pm Z$) and 8 isometric positions. (2) Render depth maps, normal maps, and silhouettes. (3) Resolution: 224×224 or 512×512 pixels. (4) Store as PNG with standardised naming. *Use case:* View-based CNNs (MVCNN), vision-language models for CAD.

8. LABELLER TRAINING PROGRAMME

CAD data labelling requires specialised domain knowledge that distinguishes it from typical image or text annotation. Labellers must understand engineering conventions, manufacturing processes, and CAD software interactions. This section details a comprehensive 50-hour training programme.

8.1 Training Curriculum

Module 1: CAD Fundamentals (10 hours)

- **Session 1.1 (2h): Introduction to CAD and B-Rep:** History of CAD, role in product development. Solid modelling concepts: CSG vs B-Rep. B-Rep data structure: faces, edges, vertices, shells, solids. Hands-on: Explore a STEP file in FreeCAD, identify faces and edges.
- **Session 1.2 (2h): Surface Types and Geometry:** Analytic surfaces: plane, cylinder, cone, sphere, torus. Free-form surfaces: B-spline, NURBS. Surface parameters: normal vectors, curvature, tangent planes. Hands-on: Identify surface types on sample parts.
- **Session 1.3 (2h): CAD File Formats:** Native vs neutral formats. STEP structure: header, data section, entity types. Common conversion issues: gaps, inverted normals, missing faces. Hands-on: Compare same part in STEP, IGES, STL.
- **Session 1.4 (2h): Manufacturing Processes Overview:** Subtractive: milling, turning, drilling, grinding. Additive: FDM, SLA, SLS, metal printing. Forming: casting, forging, stamping, injection moulding. Design for Manufacturing (DFM) principles.
- **Session 1.5 (2h): GD&T; Basics:** Datums, tolerances, feature control frames. Size tolerances, geometric tolerances. Reading engineering drawings. Relevance to feature labelling: tolerance affects feature classification.

Module 2: Machining Feature Recognition (14 hours)

- **Session 2.1 (3h): Hole Features:** Through-holes vs blind holes. Counterbores, countersinks, spot faces. Tapped holes, reamed holes. Identifying hole features: cylindrical face + axis perpendicular to stock. Edge cases: angled holes, intersecting holes.
- **Session 2.2 (3h): Pocket and Slot Features:** Rectangular, circular, irregular pockets. Through-slots vs blind slots. T-slots, dovetails. Identifying pockets: depression bounded by walls and floor. Edge cases: pockets with islands, multi-level pockets.
- **Session 2.3 (3h): Step and Passage Features:** One-sided, two-sided, blind steps. Rectangular, triangular, hexagonal passages. Identifying steps: L-shaped profile at edge. Edge cases: cascading steps, chamfered step edges.
- **Session 2.4 (3h): Transition Features:** Fillets (concave rounds at interior edges). Rounds (convex rounds at exterior edges). Chamfers (angled edge breaks). Identifying transitions: toroidal or conical face at edge intersection. Threshold rules: minimum fillet radius for labelling.
- **Session 2.5 (2h): Protrusion Features:** Bosses (cylindrical protrusions). Ribs (thin wall reinforcements). Pads (general protrusions). Identifying protrusions: material added above base surface. Distinction from pockets: inside vs outside.

Module 3: Annotation Tools and Interface (8 hours)

- **Session 3.1 (2h): 3D Viewer Navigation:** Orbit, pan, zoom controls. Standard views: front, back, top, bottom, isometric. Section views and clipping planes. Measurement tools: distance, angle, radius.
- **Session 3.2 (2h): Face Selection Techniques:** Single-click selection. Shift-click for multi-selection. Lasso selection for regions. Select all faces of same surface type. Deselect and clear selection.
- **Session 3.3 (2h): Label Assignment Workflow:** Browse taxonomy tree. Assign label to selected faces. Keyboard shortcuts for common labels. Confidence scoring. Notes and comments.

- **Session 3.4 (2h): Instance Grouping:** Create feature instance from labelled faces. Merge and split instances. Name instances for clarity. Review instance boundaries in 3D.

Module 4: Taxonomy Deep Dive (8 hours)

- **Session 4.1-4.4 (8h):** Detailed walkthrough of all 25 feature classes. Positive and negative examples for each class. Decision trees for ambiguous cases. Edge case gallery with consensus labels. Inter-annotator disagreement examples and resolutions.

Module 5: Quality Standards (4 hours)

- **Session 5.1 (2h): Common Errors:** Mislabelling (wrong feature class). Incomplete labelling (faces left as stock when they're features). Wrong instance boundaries (splitting one hole into two instances). Inconsistent labelling (same feature labelled differently in similar parts).
- **Session 5.2 (2h): Quality Metrics:** Inter-annotator agreement (Cohen's Kappa). Face-level accuracy vs instance-level IoU. Time per model benchmarks. Feedback interpretation and self-improvement.

Module 6: Practical Assessment (6 hours)

- **Assessment 6.1 (3h): Supervised Labelling:** Label 10 parts under trainer observation. Receive real-time feedback. Discuss decisions and uncertainties.
- **Assessment 6.2 (3h): Independent Labelling:** Label 10 parts independently. Results compared against gold standard. Must achieve >90% face accuracy and >0.85 Kappa to pass. Remedial training for those who don't pass.

9. PROJECT SCOPE, TIMELINE, AND SCALING

The project follows a phased approach to validate the technical framework, build operational capacity, and scale to production volumes:

Phase	Duration	Volume	Team	Key Milestones
Pilot	6 weeks	1,000 models	2 labellers, 1 engineer	Pipeline deployed, taxonomy validated, 90%+ accuracy
Scale-up	4 months	10,000-50,000	10 labellers, 2 engineers, 2 QA	Throughput >500/week, IAA >0.85
Production	Ongoing	100,000+/year	20+ labellers, 3 engineers, 3 QA	Continuous delivery, client integrations

Table 3: Project phasing and scaling plan

10. FUTURE SCOPE AND STRATEGIC OPPORTUNITIES

- **Industry Vertical Expansion:** Extend taxonomy to aerospace (structural brackets, turbine blades, aerofoils), medical devices (implants, surgical tools, prosthetics), automotive (powertrain components, body panels, interior parts), and consumer products (enclosures, buttons, connectors).
- **AI-Assisted Labelling:** Deploy pre-trained models (FeatureNet, Hierarchical CADNet) for automated pre-labelling. Labellers review and correct AI predictions rather than labelling from scratch. Expected efficiency gain: 3-5× throughput improvement after model fine-tuning.
- **Synthetic Data Generation:** Procedurally generate parametric CAD models with automatic ground-truth labels. Use CAD APIs (PythonOCC, FreeCAD) to create millions of training samples. Combine synthetic and real data for robust model training.
- **Active Learning Integration:** Implement uncertainty sampling: prioritise labelling models where AI is least confident. Reduces total labelling effort while maximising model improvement. Track model performance vs labelling cost to optimise budget allocation.
- **Multi-Modal Datasets:** Combine CAD labels with: manufacturing process data (toolpaths, machine parameters), simulation results (FEA stress, CFD flow), inspection data (CMM measurements, CT scans), cost data (material, machining time, tooling). Enable holistic AI models that understand design-to-manufacturing lifecycle.

11. IMPLICATIONS AND STRATEGIC CONSIDERATIONS

Technical Implications:

- CAD data contains proprietary design IP requiring enterprise-grade security protocols aligned with Isidor's security and privacy capabilities. All data must be encrypted at rest and in transit.
- B-Rep to mesh/voxel conversion introduces geometric approximations. Labelling should occur on original B-Rep where possible; simplified representations are for ML training only.
- Large-scale CAD datasets (100,000+ models) require significant storage (10-50TB) and compute infrastructure for preprocessing. Cloud-native architecture recommended.
- Model provenance and licensing must be carefully tracked. Some CAD models may have restrictions on use for AI training.

Operational Implications:

- CAD labelling requires higher skill levels than typical annotation, impacting workforce recruitment (engineering students, CAD technicians) and training costs (50+ hours vs 4-8 hours for image labelling).
- Taxonomy evolution based on client needs and model feedback requires flexible guidelines and periodic labeller retraining.
- Inter-annotator agreement thresholds must be established collaboratively with downstream ML teams to ensure label quality meets model requirements.
- Integration with Isidor's reasoning trace data and foundation model post-training pipelines creates unique value propositions unavailable from commodity labelling vendors.

12. CONCLUSION AND RECOMMENDED NEXT STEPS

This proposal presents a comprehensive technical framework for establishing a CAD data labelling capability aligned with Isidor's mission of transforming enterprise data into AI-ready assets. The combination of standardised file handling, rigorous preprocessing pipelines, expert labelling taxonomy, trained workforce, and quality assurance mechanisms creates a foundation for producing high-value training datasets that can power next-generation manufacturing AI applications.

By leveraging Isidor's existing capabilities in evaluation, verification, and foundation model post-training, the CAD data labelling initiative delivers unique value through tight integration with the broader enterprise AI pipeline. The phased approach allows for iterative refinement based on client feedback and model performance, ensuring continuous improvement.

Recommended Next Steps:

1. Schedule technical deep-dive meeting to review preprocessing pipeline specifications
2. Identify pilot client with 1,000+ CAD models available for labelling
3. Recruit initial labelling team (2 labellers with CAD/engineering background)
4. Set up annotation platform infrastructure and complete security review
5. Execute 6-week pilot phase and evaluate results against success criteria
6. Present pilot findings and proceed to scale-up phase upon approval

Contact:

Isidor AI

founders@isidor.ai | www.isidor.ai

San Francisco | London